Reference

README · Tutorial · **Reference**

Look things up here. Every command, file, column and rate, in one place.

Commands

`run.py`

Makes invoices and writes the answer key.

Invocation	What it does
`python run.py request`	one request per sender, asking for every document since 1 January, into `Invoices/live/outbox/`
`python run.py backlog [DATE]`	the business from 1 January to DATE (today), every document drawn into `Invoices/live/<month>/`, plus `ledger.json`, `labels.csv`, `erp/` and `clock.json`
`python run.py score`	what the reader filed in `Invoices/live/data/`, against `ledger.json`: right, flagged, wrong, missing; into `Invoices/live/score.json`
`python run.py live`	the documents issued since the last run, up to today, one mail each; never later than today
`python run.py cost`	the cost report from what was read, into `Invoices/live/cost/`
`python run.py samples [N]`	one invoice per foreign issuer and the broker, or N at random, plus `truth.json`, into `Invoices/samples/`
`python run.py unit`	each stage's tests on their own
`python run.py smoke <step>`	one smoke test of the tutorial, into `Invoices/samples/<step>/`
`python run.py batch [N]`	N invoices (100) drawn, mailed, read back and scored, into `Invoices/samples/batch/`
`python run.py monitor [SECONDS] [PORT]`	the simulated live ticks for the dashboard demonstration, into `Invoices/samples/live/`, metrics on `http://localhost:PORT/metrics` (9108)
`python run.py dataset [PROFILE] [SEED ...]`	no pages: `erp/`, `invoices.csv`, `invoice_lines.csv`, `labels.csv` and `filed/` for one run of the profile (year, busy, drift or backlog; year), into `Invoices/datasets/<profile>-seed<N>/`
`python run.py readset [PROFILE] [COUNT] [SEED]`	the same, but COUNT documents (400) drawn, mailed to a folder and read back, so the figures carry the reader's mistakes, into `Invoices/datasets/read-<profile>-seed<N>/`
`python run.py readme`	rewrite the one-run table below from the run
`--post`	with backlog or live: write the replies as `.eml` files into `Invoices/live/inbox/`
`--send`	with request, backlog or live: send them for real, after asking

Invocation	What it does
`--metrics`	with backlog: count each reply sent on `http://localhost:9110/metrics`

Nothing is dated after the run date. No invoice, receipt, arrival or discovery is drawn past it.

Smoke tests, batches and datasets write only under `Invoices/samples/` and `Invoices/datasets/`, so a test never mixes with the real run in `Invoices/live/`. `live` remembers the last day it brought in `clock.json`, so running it twice on one day sends nothing twice.

A run is seeded. The same seed gives the same documents, down to the invoice numbers. The year's events are drawn over the whole year, so a later run date only adds documents: every earlier one stays the same, which is what lets `live` carry on from the backlog.

`watch.py`

Reads a mailbox into the two CSV files.

Invocation	What it does
`python watch.py`	opens IMAP IDLE and waits for the server to push
`python watch.py --once`	takes what is already in the mailbox, then stops
`python watch.py --folder DIR`	reads a folder of `.eml` files, no server
`python watch.py --metrics PORT`	also counts each filed document on `http://localhost:PORT/metrics`

It files into `Invoices/live/data/`. Mail whose subject says `Smoke test` belongs to the tutorial's smoke steps and is left alone.

The watcher renews the connection every ten minutes, reconnects with a growing wait, and sweeps the whole folder once an hour in case a push went missing. Ctrl+C stops it; `Invoices/live/data/state.json` remembers where it got to.

Environment

Variable	Default	Used by
`INVOICE_SMTP_USER`	none, required to send	`mail/delivery.py`
`INVOICE_SMTP_TO`	the sending address	`mail/delivery.py`
`INVOICE_SMTP_HOST`	`smtp.gmail.com`	`mail/delivery.py`
`INVOICE_SMTP_PORT`	`587`	`mail/delivery.py`
`INVOICE_IMAP_HOST`	`imap.gmail.com`	`mail/watcher.py`
`INVOICE_IMAP_FOLDER`	`INBOX`	`mail/watcher.py`

The password is never an environment variable. It lives in the operating system keychain, under the service name `invoice-pipeline`, labelled with the sending address. Use an app password, never the account's own.

```
python -m keyring set invoice-pipeline you@gmail.com
```

Every issuer sends from that one address, under its own name: `"Ningbo Caplink Industrial Co., Ltd." <you+ncl@gmail.com>`. A free account cannot send from a domain it does not own.

A real backlog is several hundred mails. They go out at a steady pace, under Gmail's daily limit (Google), over more than one day if needed. `Invoices/live/sent.json` records every mail sent, so a stopped send resumes without sending one twice. When the day's limit is reached the send stops cleanly and says how many are left. A second account can send the rest: set `INVOICE_SMTP_USER` to it for that one command and store its app password; `INVOICE_SMTP_TO` keeps the mail going to the mailbox `watch.py` reads.

The request and the replies

Moty writes once to each sender:

```
Subject: Documents issued to Moty since 1 January 2026

Please send every invoice and credit note you issued to Moty since
1 January 2026, as PDF.
```

Each sender replies with its own habit, fixed per sender so a run repeats. **Every reply holds at most 10 PDFs and stays under 25 MB**, Gmail's limit for a message (Google). Attachments are PDF

only: printed by a program, or scanned.

Code	PDFs per mail	Grouped by	Replies	Subject
NCL	10	date	the same day	`Re: Documents issued to Moty since 1 January 2026`
SBP	5	month	within a week	`BRIGHTPACK INVOICES {month}`
GZV	10	kind	in two rounds	`Commercial invoices - Guangzhou Verrelia ({n}/{total})`
HCV	1	date	within a week	`Hanil // invoice {number} // due {due}`
BDT	10	month	the same day	`Fwd: invoices {month}`
ARL	5	date	in two rounds	`Aravalli Labelworks - Invoices, part {n}`
DHI	10	shipment	the same day	`DHI statement of invoices {n}`
MRK	10	kind	within a week	`Freight invoices, batch {n}`
CSM	5	date	the same day	`COSMO SHIPPING
KNG	10	month	within a week	`Rechnungen / Invoices {month}`
TLV	10	month	the same day	`Re: Documents issued to Moty since 1 January 2026`
SCA	5	shipment	within a week	`SCA - documentos de importação, lote {n}`
TVA	10	month	in two rounds	`Faturas de armazenagem {month}`

A sender that replies in two rounds sends some documents a second time. The reader files each PDF once, by its hash.

After the backlog, `live` brings each new document in a mail of its own, on the day it is issued.

Folders written

Every invoice the project makes lives in `Invoices/`. All of it is invented; none is real data.

Folder	Holds
`Invoices/samples/`	one example invoice per foreign issuer and the broker, with `truth.json`; committed on purpose
`Invoices/samples/<step>/`	each smoke test's PDFs, answer key, mail, what the reader filed and its score
`Invoices/samples/batch/`, `single/`, `live/`, `unit/`	the trial batches, the one-invoice trial, the simulated live ticks, the stages' test files
`Invoices/datasets/`	one folder per profile and seed, for a model; no PDFs
`Invoices/live/<month>/`	the real run's PDFs, one folder a month
`Invoices/live/`	`ledger.json`, `labels.csv`, `erp/`, `cost/`, `sent.json`, `clock.json`, `score.json`
`Invoices/live/outbox/`, `Invoices/live/inbox/`	the requests, and the replies as `.eml` files when `--post` is used
`Invoices/live/data/`	what the reader filed: `invoices.csv`, `invoice_lines.csv`, `state.json`

Delete `Invoices/live/` to start the real run again from an empty store.

Issuers

Thirteen bill Moty. New suppliers also join during the year.

Code	Issuer	Sells or moves	Bills in	Page	Dates	Decimals
NCL	Ningbo Caplink Industrial	closures, pumps; FOB	USD	closed grid	`2026-04-11`	`1,234.56`
SBP	Shantou Brightpack	cartons, labels; FOB	USD	tinted band	`2026-04-11`	`1,234.56`
GZV	Guangzhou Verrelia Glassworks	bottles, jars; FOB	USD	boxed form, totals on top	`2026-04-11`	`1,234.56`
HCV	Hanil Cosmetic Vessels	pumps, jars; **CIF**, so its price includes the ocean freight	USD	no rules at all	`04/11/2026`	`1,234.56`
BDT	Binh Duong Tubeworks	tubes, closures; FOB	USD	dashed rows	`11/04/2026`	`1,234.56`
ARL	Aravalli Labelworks	labels, cartons; FOB	USD	coloured rule	`11/04/2026`	`1.234,56`
DHI	DHI Global Forwarding	LCL ocean freight and air freight; local charges from its Santos office	USD, BRL	wide ledger	`11/04/2026`	`1,234.56`
MRK	Marsk Line Agencies	FCL ocean freight; local charges from its Santos office	USD, BRL	wide ledger	`2026-04-11`	`1,234.56`
CSM	COSMO Shipping Lines	FCL ocean freight; local charges from its Santos office	USD, BRL	closed grid	`2026-04-11`	`1,234.56`

Code	Issuer	Sells or moves	Bills in	Page	Dates	Decimals
KNG	Keuhne & Nager Logistics	road haulage in the small test runs; a monthly warehouse rent in the real run	USD	coloured rule	`11/04/2026`	`1.234,56`
TLV	Transportes Louveira	the truck from Santos to Moty's plant in Louveira, per container or part load	BRL	tinted band	`11/04/2026`	`1.234,56`
SCA	Santos Comissaria Aduaneira	duty statements, broker fees	BRL	no rules at all	`11/04/2026`	`1.234,56`
TVA	Terminal Valongo Alfandegado	storage in the bonded terminal	BRL	closed grid	`11/04/2026`	`1.234,56`

Page styles

Style	Table	Totals
`grid`	every cell bordered	three right aligned rows
`zebra`	filled header, alternating tints	last row on a coloured band
`banded`	coloured rule under the header	last row on a coloured band
`minimal`	no rules, grey headings	three right aligned rows
`dashed`	dashes between the rows	three right aligned rows
`form`	closed grid, totals printed above	already printed
`ledger`	double ruled, landscape	three right aligned rows

To add an issuer, add it to `business/issuers.py`, give it a lane in `business/lanes.py`, a subject line in `mail/delivery.py` and a reply habit in `mail/replies.py`. `test_render.py` fails if it has no page style; `test_operation.py` fails if it never bills.

Freight

Cargo

Every part ships in cartons: units per carton, length, width and height, and gross weight, set in `business/catalogue.py` with a source for each. A shipment's cubic metres and kilos are summed from whole cartons, so every freight line can be checked against the cargo.

Every Friday a factory ships whatever is ready that week, together, so Moty's plan allows a week for the sailing.

Mode

Mode	When	Charged on	Source
Air	a ship would arrive too late, 55% of the time, and the load is 1,500 chargeable kg or less	per chargeable kg: actual kg or m^3 × 167, whichever is greater	IATA, ÷6000
LCL, shared container	it costs less than FCL	per W/M: m^3 or tonnes, whichever is greater, minimum 1	iContainers
FCL, full container	it costs less than LCL	per container, however full	iContainers

Each sea shipment prices both, all charges included: freight, the Santos local charges, the first storage period on the CIF value, and the truck, one per container or a van for a part load. It takes the cheaper. Forwarders quote both between 8 and 15 m^3 (AB Group), but with the rates below LCL stays cheaper up to about 46 m^3 for dear goods and 60 m^3 for cheap ones, measured by `freight.all_in_usd`: at US$95 a W/M against US$4,250 for a 20', freight alone breaks even near 45 m^3, and storage on a dear cargo pulls FCL closer. So a mid-sized importer like Moty ships mostly LCL.

A load too heavy to fly, over 1,500 chargeable kg, sails late instead: above that the air bill passes the goods' own value.

FCL uses the smallest set of containers that holds both the volume and the weight:

Container	Volume	Payload
20'	33 m^3	21 t
40'	67 m^3	26 t
40' high cube	76 m^3	26 t

Source: AB Group.

Invoices and charges

Moty buys FOB, so it pays the ocean freight and everything at destination (GoFreight). HCV sells CIF: its price includes the ocean freight, so its shipments bring no freight invoice, only the destination charges.

Mode	Invoice	Issuer	Lines and basis
FCL	ocean freight, USD	MRK or CSM	freight per container: about US$5,600 to 6,000 per 40' and US$4,000 to 4,500 per 20' in 2026
FCL	local charges, BRL	the line's Santos office	terminal handling (capatazia) about R$3,700 per container; B/L release R$1,000; Siscarga US$60 per B/L
LCL	ocean freight, USD	DHI	freight per W/M, minimum 1 W/M; about US$80 to 150 per m^3 in 2026
LCL	local charges, BRL	DHI's Santos office	desconsolidação US$200 per B/L; capatazia R$80 per W/M, minimum R$150; B/L release R$1,000
Air	air freight, USD	DHI	per chargeable kg, plus fuel and security surcharges
all	storage, BRL	TVA	a share of the CIF value per period, with a minimum; FCL from 0.48% for the first 7 days, LCL 1.16% for the first period
all	duty statement and broker fee, BRL	SCA	the duties of each NCM at its tariff; the broker's fees by contract
all	truck, BRL	TLV	one truck per container for FCL; a van per trip for LCL or air

Sources: freight level S&P Global, BSI Freight and Delpa Group (China to Chile, the nearest route with an LCL figure); local charges General Noli tariff, 2025; storage Santos Brasil and Clif LCL.

The exact rate cards live in `Invoices/live/erp/rate_card.csv`.

How the reader works

The reader imports nothing from `business/issuers.py` or `drawing/layouts.py`. It is given text and works out the rest:

What it needs	How it decides
decimal mark	counts how many figures end `,00` against `.00`
date order	ISO is unmistakable; otherwise a number above twelve can only be a day
issuer	the name printed under FROM
the table	the heading row, found by its column names; it ends where prose resumes
quantity	counted from the right, at the distance the heading gives

Three questions file every arriving PDF:

```
hash already seen             -> a resend. Skip it.
number seen, hash is new      -> a revision. The old row is superseded.
neither                       -> a new document.
```

A file that cannot be read still gets a row, `unreadable`, with the reason. Nothing is dropped.

`invoices.csv`

One row per document taken from the mailbox.

Column	Meaning
`invoice_number`	as printed on the page; empty if none was found
`status`	`new`, `revised`, `superseded` or `unreadable`
`revises`	the number this one corrects, taken from its own `-R1` suffix
`issuer`	the name printed under FROM
`issue_date`, `due_date`	ISO, whatever order the page used
`subtotal`, `discount`, `grand_total`	as printed; negative on a credit note
`order_number`	the `PO-xxxxx` found anywhere on the page
`shipment_id`	the `B/L ...` or `AWB ...` found anywhere on the page
`line_count`	rows read out of the table
`decimal_mark`, `date_order`	as worked out from the page
`source`	the attachment filename
`sha256`	hash of the PDF bytes; the real identity of the document
`first_seen`	when the message arrived at the server
`registered_at`	when the row was written
`problems`	empty when nothing was wrong; "check by hand" and "OCR read" mark rows for a person
`read_by`	`text` from the PDF's text layer, or `ocr` from a scan
`bank_account`, `sender`	the account the invoice asks to be paid into, and the mail's From
`kind`	`goods`, `freight`, `credit`, `customs` (duty statement), `broker`, `local` (local charges) or `storage`
`currency`	`USD` or `BRL`
`origin`, `destination`, `container`, `gross_weight_kg`, `vessel`	the shipment strip of a freight invoice

Column	Meaning
`fx_rate`, `customs_value`	printed on a duty statement only; a rate without its four decimals is left empty
`incoterm`	the terms of a goods invoice; CIF means the freight is in the price

Status	Cause	Is it costed
`new`	a document never seen before	yes
`revised`	it corrects an invoice already on file	yes
`superseded`	an earlier row that a revision replaced	**no**
`unreadable`	the PDF could not be read	needs a person

`problems` says	What happened
`no text layer`	a scan, and Tesseract is not installed
`no invoice number`	no label matched any known spelling
`no table rows`	the heading row was never found
`lines add to X, page says subtotal Y`	the rows were read but do not reconcile
`total X is not the subtotal less the discount, Y`	a figure was misread, usually by OCR
`lines make X, page says total Y`	no subtotal was read, and the rows do not make the total
`FX rate "..." is not a rate with four decimals`	the statement's rate was misread
`issued ..., after it arrived on ...`	a date was misread: no page is issued after it lands
`no OCR reading closed; check by hand`	every OCR pass left something open; the nearest was kept

`invoice_lines.csv`

One row per line item.

Column	Meaning
`invoice_number`	joins to `invoices.csv`
`sha256`	joins to the exact document, even when numbers repeat
`row`	position in the table, from 1
`description`	the words of the row, figures removed
`quantity`, `unit_price`, `net`	the figures of the row
`code`, `hs_code`	the part number and HS code of a goods line
`weight_kg`	the weight of a freight line; empty on a surcharge

A tax line of a duty statement keeps its NCM in the description, such as `II 3923.50.00`; its `quantity` is the tax base and its `unit_price` the rate. Join on `sha256`: a revision shares its number with the document it replaces.

`cost/`

The department's report, written by `python run.py cost` from the two CSV files and the desk's records. Run it again after each reading.

File	One row per	Holds
`landed_cost.csv`	order line and part on a shipment	goods, freight, local charges, storage, truck, broker, II, AFRMM and Siscomex in reais, the landed cost and the cost per unit, the recoverable taxes apart, the rate used, `provisional` or `final`
`operations.csv`	shipment	the goods' month, mode (FCL, LCL, air, or CIF when the price carried the freight), origin, carrier, containers, kg, freight in dollars and reais, freight per kg, local charges, truck, storage, broker, duties, `provisional` or `final`
`exceptions.csv`	document a person must read	`origin` (`ours`) and the reader's reason
`kpis.json`	run	share analysed the same Brasilia day the reply arrived, our error rate, share of each month's cost known before its close

Rule	Value	Why
Rate	the duty statement's; the PTAX of the invoice date until it arrives	the declaration fixes the rate customs used
Tie	a goods invoice with no B/L on its page goes to the shipment its dock receipt names; two equal parts of one order, to the one that left on the invoice's date	the desk's records, never a guess
Split by volume	sea freight, local charges, truck, broker and AFRMM	that is how a container and shared space are charged
Split by chargeable weight	air freight and its surcharges	that is how air is charged
Split by value	storage, Siscomex; II within its NCM	storage and duty are charged on value
AFRMM	8% of the ocean freight	the rule of the tax
Credits, not cost	IPI, PIS, COFINS, ICMS	a manufacturer gets them back
Final	goods, freight (none for CIF), local charges, storage, duty statement and truck all filed	until then the cost can still move
Close	the 6th business day of the next month	the department's calendar

Brazilian import taxes

Read on 2026-10-12. Source per NCM in `business/tariffs.py`.

NCM	Parts	II	IPI	COFINS
7010.90.22	glass bottles, dropper	9%	9.75%	9.65%
3923.30.00	PET bottle	18%	15%	9.65%
3923.10.90	acrylic jar	18%	15%	9.65%
7612.10.00	aluminium tube	16%	10%	9.65%
3923.50.00	caps	18%	5%	9.65%
8424.89.90	pump, sprayer	14%	5%	10.65%
4821.10.00	labels	16%	0%	9.65%
4819.10.00	cartons	16%	15%	9.65%

PIS-Importação 2.10%; AFRMM 8% of ocean freight; Siscomex R$ 214.50; ICMS São Paulo 18%, inside its own base. Tariff changes published during a run are simulated Gecex resolutions; the PTAX series is simulated too.

`ledger.json`

The answer key for a `backlog` run.

Key	Holds
`period`	1 January and the run date
`orders`	every purchase order, with its lines and agreed prices
`shipments`	every shipment, with route, mode, dates and what it covered
`invoices`	the truth of every invoice written, with `scanned`, `anomaly`, `lookalike_of`, `pay_to` and `sender`
`chains`	order joined to goods, shipment, freight and landed cost
`counts`	totals, including how many defects were injected

A chain

```
{
  "order_number": "PO-41011",
  "factory": "HCV",
  "raised_on": "2026-02-02",
  "ordered_value": "4245.00",
  "goods_invoices": ["HCV-2026-3107"],
  "shipments": ["B/L MRK7000106"],
  "freight_invoices": ["MRK-2026-3108"],
  "freight_share": {"MRK-2026-3108": "1512.40"},
  "credit_notes": [],
  "goods_total": "1996.59",
  "freight_total": "1512.40",
  "credit_total": "0.00",
  "landed_cost": "3508.99",
  "fully_shipped": true
}
```


The chain's landed cost is in dollars: goods and international freight. The reais documents are costed by `costing/landed.py`. A shipment can carry more than one order, so each freight invoice is split between the orders aboard it. `freight_total` is the sum of this order's slices, never of the whole invoices.

Orders placed before 1 January exist, so goods arriving in January have a history. Their documents dated before 1 January are not requested, and are not in the ledger's `invoices`.

`erp/`

The records a cost desk holds besides the invoices. One CSV per table, written by `simulation/erp.py`. None carries an answer.

Table	One row per	Columns
`vendors.csv`	issuer, including those that joined mid-run	code, name, country, kind, onboarded_on, bank_account, email, terms
`vendor_changes.csv`	recorded change to the vendor master	code, field, old_value, new_value, changed_on, requested_by, verified
`purchase_orders.csv`	order	number, vendor, raised_on, required_by, value
`po_lines.csv`	order line	order_number, code, description, quantity, agreed_price
`contracts.csv`	price in force	order_number, code, version, price, effective_on
`receipts.csv`	line signed for at the dock	order_number, code, shipment_id, quantity, received_on
`shipments.csv`	shipment	identifier, vendor, carrier, mode (ocean or air), origin, destination, container (the FCL boxes, `LCL` or `LOOSE` by air), weight_kg, volume_m3, departed_on, arrived_on
`rate_card.csv`	agreed charge	kind (freight, local charge, storage, surcharge, fixed_fee), issuer, mode, basis (container, W/M, kg, CIF %), description, amount
`approval_policy.csv`	approval band	limit, approver
`items.csv`, `tariffs.csv`, `ptax.csv`	part, tariff in force, day	weights, cartons and NCMs; rates by date; the dollar rate

`labels.csv`

The answers, one row per invoice, written by `simulation/labels.py`.

Column	Meaning
`number`, `issuer`, `kind`, `issue_date`	which invoice
`anomaly`	the planted anomaly, or empty
`lookalike_of`	the anomaly this honest invoice resembles, or empty
`found_after_days`	days until a desk would find the anomaly; empty when there is none

Anomalies

Planted in the backlog and the datasets, from their own random generators, so the small test runs never change. Named in each invoice's `anomaly` key. The rates live in `simulation/anomalies.RATES`.

Anomaly	Stage	Rate	What it does	Found after, days
`price_drift`	billing	1.0% of goods	one unit price 4 to 15% above the agreed price	10 to 40
`quantity_over`	billing	0.8% of goods	one quantity 10 to 30% above what shipped	10 to 40
`round_total`	billing	0.4% of goods	an "adjustment" lifts the total to round thousands	20 to 60
`near_duplicate`	billing	0.5%	the same bill again, a new number, 1 to 4 days later	5 to 30
`split_billing`	billing	1.2% of goods between US$25,000 and US$75,000	cut into 2 or 3 pieces, each under US$25,000	30 to 90
`freight_overbill`	freight billing	7% of freight	an "Additional handling" charge, 4 to 12% of the freight	14 to 45
`duty_rate_wrong`	customs	3% of duty statements	one II line at a rate the NCM does not carry, 2 to 10 points out	15 to 45
`broker_fee_over`	customs	4% of broker invoices	one service billed 15 to 60% above its contract	30 to 90
`unmatched_vendor`	purchasing	2 to 3 companies a year	3 to 6 bills of US$3,000 to US$12,000 for vague services, no order	60 to 180

Anomaly	Stage	Rate	What it does	Found after, days
`account_mismatch`	vendor master	0.2%	an account not on the vendor master, from an address not on file	20 to 60

In a drift dataset, `freight_overbill` and `account_mismatch` double from the midpoint, and new unmatched vendors bill as freight.

Each anomaly has an honest twin, named in `lookalike_of`. Only a record tells them apart:

Twin of	What it is	The record that clears it
`price_drift`	a price rise	a version 2 line in `contracts.csv`, in force before the invoice
`quantity_over`	an over-delivery the dock accepted	`receipts.csv` holds the billed quantity
`round_total`	a mould tooling quote of 3,000, 5,000 or 8,000	`rate_card.csv`, fixed fee
`near_duplicate`	a monthly warehouse rent of US$2,000	`rate_card.csv`; every 28 days
`split_billing`	an order billed in parts, one per shipment	each part ties to its own shipment
`freight_overbill`	a peak season or imbalance surcharge	`rate_card.csv`, surcharge
`duty_rate_wrong`	a declaration applying a published rate change	`tariffs.csv` carries the new rate from its date
`broker_fee_over`	a fee the amended contract allows	the line says `(amended)`
`unmatched_vendor`	a real new supplier	`vendors.csv` and a purchase order
`account_mismatch`	the first 3 invoices after a recorded bank change	`vendor_changes.csv`, verified by call-back

Scans

Scanning is a habit of the issuer, set in `drawing/scan.HABITS`. The generator decides from the invoice number alone, so a resend of a scan is the same scan, byte for byte. Everyone else sends the PDF its system printed. Nothing on a scan can be selected or copied.

Issuer	Scans	Quality	Askew
ARL	60% of its invoices	poor	half of them, 0 to 15 degrees off upright
SCA	30%	good	never

Quality	dpi	Specks	Blur	JPEG	Colour
good	200	1 per 900 pixels	0.4	75	grey
poor	150	1 per 500 pixels	0.3	none	black and white

An askew page stays a portrait A4 sheet: its blank margins are trimmed, it is turned, and shrunk only as much as it must to sit wholly inside the reading area, 4% from each edge. At 150 dpi black and white, OCR mixes up digits such as 8 and 6, and 9 and 3.

The reader first levels a page: it finds the turn at which the rows of print run straightest, within 16 degrees, and turns it back. Then the OCR passes, tried in order until one reading closes:

Pass	dpi	Cleaning	Tesseract mode
1	300	none	3, automatic
2	300	softened, for black and white	6, one block
3	300	none	6
4	400	median filter and contrast	3
5	300	black and white	4, one column
6	400	black and white	6
7	300	none	4

A reading closes when the number, date, total, currency and lines are all there; the lines add to the subtotal; and the total is the subtotal less the discount, or, with no subtotal read, the lines make the total. When none closes, the nearest reading is kept and the row says why and "check by hand": a misread figure goes to a person, never in as right. An OCR'd number one letter away

from the attachment's name is corrected and noted. An OCR'd `||` or `ll` before an NCM is read as `II`, the import duty. Tesseract is found on `PATH` or at `C:\Program Files\Tesseract-OCR`.

Defects and distributions

Defect	Rate	Where
Resend	6%	`defects.DUPLICATE_CHANCE`
Revision	5%	`defects.REVISION_CHANCE`
Credit note	4%	`defects.CREDIT_CHANCE`
Wrong order quoted	7%	`defects.WRONG_ORDER_CHANCE`
Partial shipment	15%	`operation.PARTIAL_CHANCE`
Demurrage on KNG's truck, in the runs without Brazil	18%	`operation.DEMURRAGE_CHANCE`

Quantity	Shape	Set in
Unit price	lognormal around the catalogue price	`pricing.unit_price`
Supplier bias	fixed per issuer, 0.83 to 1.21	`business/issuers.py`
Spread	fixed per issuer, 0.06 to 0.19	`business/issuers.py`
Volume discount	four steps, 1.00 down to 0.86	`pricing._volume_discount`
Order size	2 to 60 packs, weighted to the middle	`pricing.order_quantity`
Freight rate	the rate card, fixed per container, W/M or kg; KNG's road trip lognormal around US$1,420	`simulation/freight.py`, `pricing.freight_rate`
Surcharge	a band per named charge	`pricing.SURCHARGE_BANDS`
Lead time	35 to 55 days, plus a slip of 0, 3, 7 or 14	`operation._lead`
Transit	30 to 46 days by sea, 3 to 6 by air	`lanes.LANES`
Reply delay	the same day, within a week, or two rounds	`mail/replies.py`
Late order flies	55% of the time	`operation.AIR_CHANCE_WHEN_LATE`

A unit price keeps three or four decimals when it is under a dollar.

Metrics

Served by `python run.py monitor` on `/metrics`, read by Prometheus.

Metric	Kind	Labels	Meaning
`invoices_total`	counter	`stage`: drawn, mailed, read	invoices through each stage
`documents_by_status_total`	counter	`status`	what the reader filed each new document as
`stage_seconds`	histogram	`stage`: draw, mail, read	time each stage of a tick took
`read_accuracy_ratio`	gauge		share of the last tick's totals read right
`stored_documents`	gauge		rows in `invoices.csv`
`simulated_day_timestamp_seconds`	gauge		noon UTC of the day the last tick released
`last_tick_success`	gauge		1 when the last tick finished, 0 when it failed
`tick_failures_total`	counter		ticks that raised an error
`documents_read_total`	counter	`method`: text, ocr	new documents, by how they were read
`documents_flagged_total`	counter		new documents left for a person
`analysed_same_day_ratio`	gauge		documents registered on the Brasilia day they arrived
`our_error_ratio`	gauge		documents the reader could not read or close
`documents_by_kind_total`	counter	`kind`	new documents by kind
`registration_delay_seconds`	histogram		from the mail arriving to its row being written
`landed_cost_brl`	gauge	`status`: provisional,	the cost report's total

Metric	Kind	Labels	Meaning
		final	
`mails_sent_total`	counter	`sender`	replies sent to the real mailbox by `backlog --send --metrics`

The real run reports from the host: `watch.py --metrics 9108` counts each document it files, `backlog --send --metrics` counts each reply on 9110. Prometheus scrapes both at `host.docker.internal`. Start only the dashboard for it: `docker compose up -d prometheus grafana`.

Project 02 serves four more on port 9109: `anomaly_flags` by stage, `anomaly_decisions` by decision, `anomaly_flagged_amount` and `anomaly_champion_average_precision`.

Service	Port	Does
`pipeline`	9108	`python run.py monitor 15`
`prometheus`	9090	every 15 seconds: `pipeline:9108`; on the host, the watcher on 9108, the sender on 9110 and project 02 on 9109
`grafana`	3000	the dashboard in `grafana/dashboards/pipeline.json`, no login

All three start with `docker compose up -d --build` inside `monitoring/`. For the real run, start only `prometheus grafana`. From WSL, with Python running on Windows, pass the Windows address as the host:

```
PIPELINE_HOST=$(ip route | awk '/default/ {print $3}') docker compose up -d prometheus graf
```

One run

Written by `python run.py readme`. A test fails if this table drifts from the run.

One run: 8 weeks from 2026-04-06, seed 2026.

Purchase orders raised	1037
Shipments that left a port	89
Invoices in the period	960
Goods invoices	642
Freight invoices	102
Credit notes	14
Duty statements, in reais	51
Broker invoices, in reais	51
Resends: same number, same bytes	47
Revisions, `-R1`	50
Invoices quoting the wrong order	48
Chains with a landed cost	430
International freight as a share of goods	about 5%
Arriving as scans	79
Anomaly: broker fee over	1
Anomaly: freight overbill	7
Anomaly: near duplicate	4
Anomaly: price drift	6
Anomaly: quantity over	4
Anomaly: round total	1
Anomaly: split billing	5
Anomaly: unmatched vendor	2
Honest look-alike of broker fee over	2
Honest look-alike of freight overbill	4
Honest look-alike of near duplicate	2
Honest look-alike of price drift	6

Honest look-alike of quantity over	6
Honest look-alike of round total	2
Honest look-alike of split billing	15
Honest look-alike of unmatched vendor	1

What the reader scored

Every document against `ledger.json`. The table below was measured on 2026-10-15, by `python run.py score` on the real run: all 1,003 documents of the period, 1 January to 14 October, read from Gmail.

Arrives as	Number, total, kind and currency right	Flagged for a person	Wrong and not flagged
PDF with text	900 / 900	0	0
Scan, good (SCA, 200 dpi)	82 / 82	0	0
Scan, poor (ARL, 150 dpi, black and white)	8 / 8	0	0
Scan, poor and askew (ARL)	2 / 13	11	0
All	**992 / 1,003**	**11**	**0**

Files

```
01-invoice-pipeline/
├── run.py                  - makes invoices, in any mode
├── watch.py                - reads the mailbox
├── Dockerfile              - runs the smoke test
├── monitoring/             - Prometheus and Grafana, as code
├── Invoices/               - every invoice the project makes
│   ├── samples/            - examples, and the smoke tests' steps
│   ├── datasets/           - figures for a model, no PDFs
│   └── live/               - the real run
│
├── src/
│   ├── business/           - the world being billed
│   │   ├── buyer.py        - Moty, the one billed
│   │   ├── catalogue.py    - the parts Moty buys, and their cartons
│   │   ├── invoice.py      - the invoice's sums
│   │   ├── issuers.py      - the thirteen who bill
│   │   ├── lanes.py        - port, carrier, days
│   │   ├── orders.py       - order and shipment
│   │   ├── profiles.py     - who each sender is, for smoke test 2
│   │   └── tariffs.py      - Brazilian import taxes
│   │
│   ├── simulation/         - making it happen
│   │   ├── anomalies.py    - errors planted for a model
│   │   ├── builder.py      - one invoice alone
│   │   ├── customs.py      - duty statements, broker, truck
│   │   ├── defects.py      - resends, revisions
│   │   ├── erp.py          - the desk's records
│   │   ├── freight.py      - FCL, LCL or air, and every charge
│   │   ├── fx.py           - the dollar rate, by day
│   │   ├── labels.py       - the answers, one row each
│   │   ├── ledger.py       - the answer key
│   │   ├── lookalikes.py   - honest twins of anomalies
│   │   ├── newcomers.py    - vendors that join mid-run
│   │   ├── operation.py    - the day-by-day clock
│   │   └── pricing.py      - where numbers come from
│   │
│   ├── drawing/            - the page
│   │   ├── formatting.py   - numbers and dates
│   │   ├── layouts.py      - each issuer's page
│   │   ├── parts.py        - shared pieces
│   │   ├── render.py       - invoice to PDF
│   │   ├── scan.py         - ARL's and SCA's scans
│   │   └── styles.py       - the seven designs
│   │
│   ├── mail/               - the mailbox
│   │   ├── request.py      - Moty's request to each sender
│   │   ├── replies.py      - each sender's batches
│   │   ├── delivery.py     - sending over one connection
│   │   ├── sources.py      - folder or account
```

```
│   │   └── watcher.py      - IMAP IDLE
│   │
│   ├── reading/            - reading it back
│   │   ├── fields.py       - text to fields
│   │   ├── intake.py       - two CSV files
│   │   └── ocr.py          - reading the scans
│   │
│   ├── costing/            - what each part cost
│   │   ├── landed.py       - the landed cost report
│   │   ├── kpis.py         - the desk's targets
│   │   └── truth.py        - a perfect reader, for checks
│   │
│   ├── runs/               - the modes
│   │   ├── backlog.py      - the real run: request, backlog, live, score
│   │   ├── modes.py        - batch, dataset, the simulated ticks
│   │   ├── report.py       - the one-run table
│   │   └── smoke.py        - the tutorial's seven smoke steps
│   │
│   └── monitoring/         - watching it run
│       └── metrics.py      - what the dashboard reads
│
└── tests/                  - the same eight folders
```

Tests

630 tests, in the same eight folders as `src/`, plus one that checks every link in the docs. The full suite takes about five minutes. Tests marked `ocr` run Tesseract on real scans; tests marked `scans` draw the scans without reading them.

```
python -m pytest                        # everything
python -m pytest tests/costing -q       # one folder
python -m pytest -k freight -q          # anything about freight
python -m pytest -m "not ocr"           # skip the slow OCR tests
python run.py unit                      # one folder at a time
```

Sources

1. iContainers, "How to Calculate Ocean Freight Charges". https://www.icontainers.com/help/how-to-calculate-ocean-freight-charges/
2. AB Group Shipping, "USA-Latin America FCL vs LCL: The 10 CBM Breakpoint". Container volumes and payloads; where FCL beats LCL. https://www.abgroupshipping.com/blogs/en/fcl-vs-lcl-for-exporters-when-to-choose-full-container-on-usa-latin-america-rout

3. Airlift USA, "Chargeable Weight". The IATA ÷6000 rule, 167 kg per m^3. https://airliftusa.com/glossary/chargeable-weight
4. GoFreight, "FOB Incoterms Explained". Who pays what under FOB. https://gofreight.com/blog/industry-insights/mastering-fob-in-shipping-a-complete-guide.html
5. S&P Global, "Container rates to South America stay elevated amid disruptions", August 2026. https://www.spglobal.com/energy/en/news-research/latest-news/shipping/081126-container-rates-to-south-america-stay-elevated-amid-disruptions
6. General Noli, "Taxas locais - Brasil", August 2025. https://www.generalnoli.com/wp-content/uploads/2025/09/Brazil-local-charges-internet-2025.pdf
7. Santos Brasil, Tecon Santos price table, 2017. https://www.santosbrasil.com.br/downloads/tabela_precos/Tabela_Precos_2017-Tecon_Santos.pdf
8. Clif, "Tabela de preços e serviços LCL", 2024. https://oclif.com.br/wp-content/uploads/2024/09/Tabela-de-precos-e-Servicos-LCL-Clif-2024.pdf
9. BSI Freight, "Shipping from China to Brazil", 2026. 20GP US$4,000 to 4,500. https://www.bsifreight.com/shipping-routes/shipping-from-china-to-brazil
10. Delpa Group, "China-Chile Maritime Freight Cost 2026". LCL US$80 to 150 per m^3. https://www.delpagroup.com/en/guias/costo-flete-maritimo-china-chile
11. Google, Gmail attachment and sending limits. https://support.google.com/mail/answer/6584, https://support.google.com/mail/answer/22839
12. APQC, "Metric of the Month: Detect and Prevent Duplicate or Erroneous Payments", CFO.com. The basis for the billing error rates. https://www.cfo.com/news/metric-of-the-month-detect-and-prevent-duplicate-or-erroneous-payments/656852/
13. Tompkins Ventures, "Why Freight and Parcel Shipping Require Post-Auditing". The basis for the freight error rate. https://tompkinsventures.com/why-freight-and-parcel-shipping-require-post-auditing/